

Constructor :

Constructor is a member function of a class.

The name of the constructor is same as the name of the class.

It has no return type, so can't use return keyword.

Constructor can never be static.

- A Constructor is a special member function whose task is to initialize the objects of its class.
- ★ Constructor is not called it is implicitly invoked when an object is created.

Program: `#include <iostream>`

`using namespace std;`

`class complex`

`{`

`private:`

`int a, b;`

`public:`

`complex()`

`{`
`cout << "Hello from constructor";`

`}`

`};`

`void main()`

`{`

`complex c1; // Hello from constructor`

`complex c2, c3; // Hello from constructor Hello from constructor`

`}`

Important →

Constructors are crucial for ensuring that objects are correctly initialized with valid data from the moment of creation, preventing issues with uninitialized members.



* Important :

If in any program object is created the constructor is called by default by a compiler.

If we don't make an **constructor** the compiler by - default formed an constructor with zero **parameters** and there is **no coding** inside its body.

This type of constructor is known as **default constructor**.
example :-

```
class name ( )  
{  
}  
}
```

And Once you make an constructor by yourself in a program then the compiler don't make any **default** constructor by itself.

Program: #include <iostream>
using namespace std;

class complex

{

private:

int a, b;

public:

complex(int x, int y)

{

a = x;

b = y;

}

};

void main()

{

complex c1(3, 4), c2; // object c1 with two argument is declared

so the constructor taking two argument is by-default called by the compiler

But there is no constructor with zero parameter for object c2.

So program will cause an **error**.



Program: #include <iostream>

using namespace std;

class complex

{

private:

int a, b;

public:

complex(int x, int y) // Parameterized Constructor

{

a = x;

b = y;

complex(int k)

{

a = k;

complex()

{

a = 0;

b = 0;

void main()

{

complex c1(3,4) or complex c1 = complex(3,4);

complex c2(5) or complex c2 = complex(5) or complex c2 = 5;

complex c1 = complex(3,4), c2 = 5, c3, c4(3,8);

}



As we discuss earlier that if we don't declare any constructor in an program the compiler by-default formed an default constructor.

And the compiler also formed/declare another constructor known as copy constructor.

Important →

The compiler by-default formed two constructor **default constructor** and **copy constructor**.

If we make default constructor the compiler don't formed default constructor but the compiler by-default formed copy constructor.

If we make copy constructor by ourself then the compiler don't formed any type of constructor by itself.

```
complex c4 = c1; } Copy constructor call  
complex c4(c1);
```

Important ÷

The compiler do not form any ~~the~~ parameterized constructor by itself. it only form/create **default & copy constructor**.

And if we create an parameterized constructor then no **default constructor** is created by-default but the compiler by default formed **copy constructor**.



Program: #include <iostream>

using namespace std;

Class complex

{

private:

int a, b;

public:

Complex(int x, int y)

{

a = x;

b = y;

Complex(int k)

{

a = k;

}

Complex()

{

}

Complex(Complex &c) → Copy constructor

{

a = c.a;

b = c.b;

}

};

void main()

{

Complex c1(3, 4), c2, c3(5);

Complex c4(c1);

}